CPF All-in-One Dashboard

Full Product Specification

For use with Claude Code. Covers all CPF pillars: OA / SA / MA / RA, retirement planning, housing, Medisave, CPF LIFE, tax relief, and growth strategy optimisation.

Target users: Financial advisors & HR teams

Platform: Next.js 14 web application (manual data entry, no Singpass)

1. Project Summary

A full-stack CPF planning dashboard for financial advisors and HR teams. Users manually input client CPF data and receive deterministic, time-series projections across all CPF pillars. All computation runs as a **stateful financial state machine** — every year is simulated step by step, with policy rules applied in the correct sequence.

2. Tech Stack

Layer	Choice
Frontend	Next.js 14 (App Router), React, TypeScript, Tailwind CSS, shadcn/ui
Charts	Recharts
Backend	Python FastAPI
Database	PostgreSQL
State (client)	Zustand
Auth	None (MVP)

3. Data Model — Member Profile

```
MemberProfile {
  name: string                  // label only
  dob: date                     // derives age, years to 55/65
  monthlyGrossWage: number
  employmentStatus: 'employee' | 'self-employed'
  currentBalances: {
    OA: number
    SA: number
    MA: number
    RA: number                  // 0 until age 55
  }
  housingData?: HousingRecord
  voluntaryTopUps?: TopUpRecord[]
}
```

4. CPF Constants & Policy Engine

All rates and thresholds must live in a **single versioned policy table** in PostgreSQL. No hardcoding in business logic. Admin approval required before any policy snapshot becomes active. Version history must be retained.

```
PolicySnapshot {
  effectiveYear: number
  FRS: number
  BRS: number                   // FRS / 2
  ERS: number                   // FRS x 1.5
  BHS: number
  cpfLifeEligibilityMin: number // $60,000 RA
  ordinaryWageCeiling: number   // $6,800/month (2025)
  additionalWageCeiling: number // $102,000/year
  contributionRates: ContributionRateTable
  allocationRates: AllocationRateTable
}
```

5. Contribution Rate Table

Employee + employer combined, by age band:

Age Band	Total Rate
≤35	37%
35–45	37%
45–50	36%
50–55	35%
55–60	31%
60–65	22%
65–70	16.5%
>70	12.5%

6. Allocation Rate Table

Percentage of total CPF contribution routed to each account, by age band:

Age Band	OA	SA	MA
≤35	62.17%	16.21%	21.62%
35–45	59.69%	18.69%	21.62%
45–50	51.36%	21.62%	27.02%
50–55	40.55%	31.08%	28.37%
55–60	35.30%	33.82%	30.88%
60–65	14.00%	44.00%	42.00%
65–70	6.07%	30.30%	63.63%
>70	8.00%	8.00%	84.00%

7. Interest Rates

Account	Base Rate	Extra (first $60k combined)
OA	2.5%	+1%
SA	4.0%	+1%
MA	4.0%	+1%
RA	4.0%	+2% (first $30k)

Extra interest applies on combined balance, prioritised RA → SA/MA → OA. Computed monthly, credited annually.

8. Core Flow Engine — Contribution Routing

Execute in this exact sequence every simulated month:

Step 1 — Compute gross contribution

```
totalCPF = monthlyWage × totalContributionRate(age)
```

Step 2 — Allocate to OA / SA / MA

```
OA_contrib = totalCPF × allocationRate.OA(age)
SA_contrib = totalCPF × allocationRate.SA(age)
MA_contrib = totalCPF × allocationRate.MA(age)
```

Step 3 — MA Overflow Rule

```
IF MA_balance < BHS:
    MA += MA_contrib
ELSE:
    SA += MA_contrib         // MA overflow → SA
```

Step 4 — SA Overflow Rule

```
IF SA_balance < FRS(currentYear) AND NOT FRS_achieved:
    SA += SA_contrib
ELSE:
    OA += SA_contrib         // SA overflow → OA
```

Step 5 — Post-FRS Achievement Rule (Critical)

FRS_achieved is set to TRUE the **first time** SA_balance >= FRS(t). Once set:

- MA overflow routes to OA only (not SA)
- SA continues to earn 4% independently
- FRS_achieved flag is never unset, even if SA later dips below FRS

```
IF FRS_achieved == TRUE:
    MA overflow → OA (not SA)
```

Step 6 — Add OA_contrib to OA

```
OA += OA_contrib
```

9. Age 55 Retirement Account Engine

Triggered on the member's 55th birthday as a one-time atomic event:

- **Step 1** — Create RA
- **Step 2** — Transfer SA → RA first, then OA → RA until RA == FRS (or balances exhausted)
- **Step 3** — Record residual OA, SA, and RA_formed
- **Step 4** — Apply applicable scenario modifier (see Section 11)

```
Transfer SA → RA until RA == FRS
If SA exhausted and RA < FRS:
    Transfer OA → RA until RA == FRS (or OA exhausted)

OA_leftover = OA - amount_transferred_from_OA
SA_leftover = SA - amount_transferred_from_SA
```

```
RA_formed   = total transferred
```

10. CPF LIFE Engine

Eligibility

- Born ≥ 1958
- RA ≥ $60,000 at payout start age

Plans

Plan	Payout Behaviour
Standard	Level monthly payout; bequest from unused premium
Escalating	+2% annual payout growth; lower starting payout
Basic	Declining payouts; larger bequest

Deferral Rule

- Default payout age: 65
- Deferral allowed up to age 70
- +7% monthly payout per year deferred (max +35% at age 70)

Projection Outputs

```
CPFLIFEProjection {
  monthlyPayout: number
  annualPayout: number
  lifetimePayout: number       // to age 90 (default longevity)
  breakEvenAge: number
  deferralBonusApplied: number // % uplift from deferral
}
```

11. Retirement Scenarios Engine

Scenario A — Below BRS

- Member RA < BRS at age 55
- Output: reduced CPF LIFE payout estimate, shortfall amount, top-up recommendation

Scenario B — Property Pledge

- Condition: HDB lease covers member to at least age 95
- Allows BRS instead of FRS as RA target
- Output: freed OA/SA balance, payout comparison vs full FRS

Scenario C — ERS Optimisation

```
ERS_topup_needed = ERS - RA_balance
payout_uplift    = CPFLIFE_payout(ERS) - CPFLIFE_payout(FRS)
tax_relief_eligible = min(ERS_topup_needed, $8,000)
```

12. Tax Relief Engine

Relief Type	Cap
RSTU (self)	$8,000
RSTU (family top-ups)	$8,000
Combined cap	$16,000

Eligible transactions: RSTU, voluntary CPF contributions, family top-ups (SA/RA).

Output per transaction: taxReliefEarned, remainingCap, estimatedTaxSaved (based on declared tax bracket).

13. Growth Strategy Engine

Surface these as ranked recommendations on the Optimisation Dashboard:

Strategy	Trigger Condition	Output
ERS Top-Up	RA < ERS at 55	top-up amount, payout uplift, tax relief
MRSS Scheme	Age 55–70, eligible	matching amount, govt co-contribution
CPF Deferral	Payout age < 70	+7%/yr uplift, break-even comparison
Voluntary Contributions	SA/MA < ceiling	tax efficiency gain, compounding projection

14. Dashboard Modules

14.1 Main Dashboard

- Total CPF Net Worth (OA + SA + MA + RA)
- Account breakdown — current + projected
- Projected CPF at ages 55 / 65 / 90
- Retirement readiness score (0–100, rule-based)

14.2 Milestones Dashboard

- Age at which BHS achieved
- Age at which FRS achieved
- Age at which ERS achieved (if on track)
- CPF LIFE eligibility age
- Visual timeline (Recharts)

14.3 Optimisation Dashboard

- Recommended top-up amount
- Tax savings opportunity
- Fastest path to FRS / ERS
- Account routing optimisation suggestions

14.4 Housing Module

- CPF used for housing (principal drawn)
- Accrued interest owed back to CPF

- Remaining CPF housing withdrawal limit
- Projected impact on retirement adequacy

14.5 Medisave & MediShield Life Module

- Current MA balance vs BHS
- MediShield Life premium schedule by age
- MA adequacy projection to age 85
- Surplus / shortfall indicator

15. AI Policy Update System

Supports ingestion of: CPF Board PDFs, Budget Statements, Board Circulars, Annual Reports.

Extraction targets: BHS, FRS, ERS, BRS, allocation rates, contribution rates, CPF LIFE rule changes.

Workflow

- Admin uploads document
- AI extracts structured policy fields
- Preview diff vs current active snapshot
- Admin approves → new PolicySnapshot created with effective year
- All simulations re-run against new snapshot

16. Simulation Principle

> Every simulation runs as a **deterministic, stateful, year-by-year (monthly-tick) time series**. Each month: apply contributions → apply overflow rules → apply interest → check milestone flags → record state. No shortcuts. No lump-sum approximations.

```
for year in range(current_age, 100):
    for month in range(12):
        policy = get_policy_snapshot(year)
        state  = apply_contributions(state, policy)
        state  = apply_overflow_rules(state, policy)
        state  = apply_interest(state, policy)
        state  = check_milestones(state, policy)
        timeline.append(deepcopy(state))
```

17. Future Features (Backlog — Do Not Build Now)

- Monte Carlo simulation (salary growth, interest rate variance)
- Inflation-adjusted projections
- Couple CPF joint planning
- HDB mortgage CPF integration
- Healthcare cost modelling
- Retirement withdrawal simulation
- Policy change impact simulator
- AI retirement advisor chatbot

18. Key Engineering Constraints

1. Policy rules must never be hardcoded — always read from the versioned PolicySnapshot table.
2. FRS is dynamic — it grows annually; always use FRS(year), not a static value.
3. FRS_achieved is a one-way flag — once set, never unset within a simulation run.
4. Overflow order matters — MA → SA/OA must be resolved before SA → OA in every tick.
5. Age 55 RA transfer is a one-time atomic event — simulate as a single transaction, not spread across months.

Key System Insight

CPF is a financial state machine with evolving policy rules and time-based transitions. Every engine must respect the simulation tick order and the one-way flags that govern account routing behaviour.